

Part A — malloc(), calloc(), realloc(), free()

Question

Write a C program to demonstrate dynamic memory allocation using malloc(), calloc(), realloc() and free(). Use Valgrind to check for memory leaks.

Copy-on-Write using fork()

Part B

Question

Write a C program to demonstrate Copy-on-Write (COW) after fork(). The parent and child should initially share memory, and the child should modify the data to demonstrate that a separate copy is created.

```
    printf("\nChild PID: %d\n", getpid());

    printf("Child before modification: %d\n",
           data[0]);

    data[0] = 999;

    printf("Child after modification: %d\n",
           data[0]);

    printf("Copy-on-Write demonstrated.\n");

    free(data);

    exit(0);
}

else {

    wait(NULL);

    printf("\nParent after child terminates:\n");

    printf("Parent data[0] = %d\n",
           data[0]);

    printf("Parent retains original value.\n");

    free(data);
}

return 0;
}
```

saikrishna__90156@DESKTOP-2JF9H5T:~\$ |

```

        free(a);
        free(b);

        printf("\nMemory released successfully.\n");

        return 0;
    }
}
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat Practical8B.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

int main() {

    int *data;

    data = malloc(SIZE * sizeof(int));

    if (data == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < SIZE; i++) {
        data[i] = 10;
    }

    printf("Parent PID: %d\n", getpid());

    printf("Original data[0] = %d\n", data[0]);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0) {

        printf("\nChild PID: %d\n", getpid());

        printf("Child before modification: %d\n",

```

```
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        a[i] = (i + 1) * 10;
    }

    printf("malloc values:\n");

    for (int i = 0; i < 5; i++) {
        printf("%d ", a[i]);
    }

    printf("\n");

    // calloc
    int *b = calloc(5, sizeof(int));

    if (b == NULL) {
        printf("calloc failed.\n");
        free(a);
        return 1;
    }

    printf("\ncalloc values:\n");

    for (int i = 0; i < 5; i++) {
        printf("%d ", b[i]);
    }

    printf("\n");

    // realloc
    a = realloc(a, 10 * sizeof(int));

    if (a == NULL) {
        printf("realloc failed.\n");
        free(b);
        return 1;
    }

    for (int i = 5; i < 10; i++) {
        a[i] = (i + 1) * 10;
    }

    printf("\nAfter realloc:\n");

    for (int i = 0; i < 10; i++) {
```

```
saikrishna_90156@DESKTOP-2JF9H5T- X Windows PowerShell X + v

    return 0;
}
saikrishna_90156@DESKTOP-2JF9H5T:~$ nano Practical8A.c
saikrishna_90156@DESKTOP-2JF9H5T:~$ gcc Practical8A.c -o Practical8A
saikrishna_90156@DESKTOP-2JF9H5T:~$ ./Practical8A
malloc values:
10 20 30 40 50

calloc values:
0 0 0 0 0

After realloc:
10 20 30 40 50 60 70 80 90 100

Memory released successfully.
saikrishna_90156@DESKTOP-2JF9H5T:~$ nano Practical8B.c
saikrishna_90156@DESKTOP-2JF9H5T:~$ gcc Practical8B.c -o Practical8B
saikrishna_90156@DESKTOP-2JF9H5T:~$ ./Practical8B
Parent PID: 4736
Original data[0] = 10

Child PID: 4737
Child before modification: 10
Child after modification: 999
Copy-on-Write demonstrated.

Parent after child terminates:
Parent data[0] = 10
Parent retains original value.
saikrishna_90156@DESKTOP-2JF9H5T:~$ cat Practical8A.c
#include <stdio.h>
#include <stdlib.h>

int main() {

    // malloc
    int *a = malloc(5 * sizeof(int));

    if (a == NULL) {
        printf("malloc failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        a[i] = (i + 1) * 10;
    }
}
```

```
saikrishna__90156@DESKTOP- X Windows PowerShell X + v
saikrishna__90156@DESKTOP-2JF9H5T:~$ nano Practical6.c
saikrishna__90156@DESKTOP-2JF9H5T:~$ gcc Practical6.c -o Practical6
saikrishna__90156@DESKTOP-2JF9H5T:~$ ./Practical6
Server started.
Waiting for client...
Client: hello sir
exit
^C
saikrishna__90156@DESKTOP-2JF9H5T:~$ cat Practical6B.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIF01 "client_to_server"
#define FIF02 "server_to_client"

int main() {
    char message[200];
    char response[200];

    printf("Client started.\n");

    while (1) {

        printf("Enter message: ");

        fgets(message, sizeof(message), stdin);

        message[strcspn(message, "\n")] = '\0';

        int fd1 = open(FIF01, O_WRONLY);

        write(fd1, message, strlen(message) + 1);

        close(fd1);

        if (strcmp(message, "exit") == 0) {
            break;
        }

        int fd2 = open(FIF02, O_RDONLY);

        read(fd2, response, sizeof(response));

        close(fd2);
```